IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts Science Commerce and Management

High Level Design

on

Identity Security Framework For Smart city

Student Name: Ganga N S

Industry Mentor: Shashank Koram **Guided by: Vikyath Shetty**

Table of Contents

1. Introduction 3
1.1. Scope of the document 3
1.2. Intended Audience 3
1.3. System overview 3
2. System Design 4
2.1. Application Design 4
2.2. Process Flow 4
2.3. Information Flow 5
2.4. Components Design 5
2.5. Key Design Considerations 6
2.6. API Catalogue 6
3. Data Design 6
3.1. Data Model 6
3.2. Data Access Mechanism 7
3.3. Data Retention Policies 7
3.4. Data Migration 7
4. Interfaces 8
5. State and Session Management 8
6. Caching 8
7. Non-Functional Requirements 8
7.1. Security Aspects 8
7.2. Performance Aspects 9
8. References 9

1. Introduction

This document provides a high-level architectural overview of the **Identity Security Framework for Smart City**. It outlines the system structure, components, workflows, and design considerations for building a secure authentication system using Python, Flask, and SQLite.

The system focuses on secure identity management by implementing authentication and basic authorization mechanisms suitable for a Smart City environment.

1.1 Scope of the Document

This document covers:

- **User Authentication**: Secure login system using hashed passwords
- **User Registration**: Signup functionality with validation
- **Session Management**: Handling user sessions using Flask-Login
- **Authorization**: Basic role-based access (user/admin)
- **Data Storage**: SQLite database for storing user credentials

1.2 Intended Audience

- **Developers**: To understand system architecture and implementation flow
- **Cyber Security Students**: To learn authentication workflows
- **Database Handlers**: To understand schema and storage design.

1.3 System Overview

The system is a web-based authentication platform designed for Smart City applications. It allows users to:

- Register securely
- Log in using credentials
- Maintain authenticated sessions
- Access protected resources

Admins can monitor and manage users.

2. System Design

2.1 Application Design

The system follows a simple layered architecture:

- **Frontend Layer** ○ HTML forms for Signup/Login
  - Browser-based interaction
- **Backend Layer (Flask)** ○ Handles HTTP requests and responses ○ Performs validation and authentication
  - Manages sessions using Flask-Login
- **Database Layer (SQLite)**
  - Stores user credentials securely
  - Lightweight and file-based database

2.2 Process Flow

Signup Process

1. User enters name, email, and password
2. Input validation is performed
3. Password is hashed securely
4. Data is stored in SQLite database
5. User is redirected to login page

Login Process

1. User enters email and password
2. System fetches stored user data
3. Password hash is verified
4. Session is created using Flask-Login
5. User is redirected to dashboard

Logout Process

1. Session is terminated
2. User is redirected to login page

2.3 Information Flow

- **Storage Phase**: SQLite stores/retrieves user data
- **Response Phase**: **Input Phase**: User submits form data via browser
- **Processing Phase**: Flask backend validates and processes data
- System sends response and updates UI

2.4 Components Design

1. Authentication Module

- Handles login and signup
- Password hashing and verification

2. User Management Module

- Stores user data
- Manages user roles and status

3. Session Management Module

- Maintains login sessions
- Uses Flask-Login

4. Database Module

- SQLite database
- CRUD operations for users

2.5 Key Design Considerations

- **Security First Approach** ○ Password hashing (not plain text) ○ Input validation
- **Minimal Dependencies** ○ Uses only Flask, Flask-Login, SQLite
- **Scalability** ○ Designed for extension (future Smart City services)
- **Simplicity** ○ Easy to understand for learning purposes

2.6 API Catalogue

Endpoint	Method	Description
/signup	GET, POST	Register new user
/login	GET, POST	Authenticate user
/logout	GET	Logout user
/dashboard	GET	Protected user page

3. Data Design

3.1 Data Model

User Table

Field Type Description id

Integer Primary Key name

Text User name email Text

Unique email

password_hash Text Hashed password

role Text User/Admin

status Boolean Active/Inactive

3.2 Data Access Mechanism

- Uses Python sqlite3 module
- Parameterized queries to prevent SQL injection
- Data retrieved using row-based access

3.3 Data Retention Policies

- User data stored permanently unless deleted
- Admin can remove inactive users
- No automatic deletion policy implemented.

3.4 Data Migration

- **Initial Provisioning:** The process involves creating the initial database.db file and executing SQL schema scripts to establish the **User** table.
- **Schema Evolution:** Any future updates to the security framework, such as adding multi-factor authentication fields, will require migrating existing resident records to the updated SQLite table structure while maintaining data integrity.

- **Secure Transition:** During development cycles, data migration ensures that user credential hashes are moved from local testing environments to the deployment environment without exposing sensitive plain-text data.

4. Interfaces

- Frontend: A browser-based UI using HTML5 and CSS.

5. State and Session Management

- State Management: Secure, signed Cookies are used via Flask-Login to track the current_user.

6. Caching

- **Session Caching:** Instead of querying the database every time a user clicks a new page to verify they are logged in, the Flask-Login session (stored in signed cookies) acts as a form of client-side cache for the user's identity.
- **Static Asset Caching:** Your browser caches the CSS and images used in your "Sign Up" and "Login" forms so the page loads instantly on repeat visits.
- **Database Query Caching:** Frequent lookups, such as checking if a resident's account is "Active" or "Disabled," can be cached to reduce the load on the database.db file.

7. Non-Functional Requirements

7.1 Security Aspects

- Cryptography: All passwords are hashed with high-iteration PBKDF2 encryption.

- SQL Safety: Parameterized queries are used for all database actions to prevent SQL Injection.

7.2 Performance Aspects

- High Speed: Lookups usually take less than 1 millisecond due to the local SQLite structure.
- Lightweight: Low memory footprint suitable for urban management hardware.

8. References

- Flask Documentation: https://flask.palletsprojects.com/
- Python sqlite3 Documentation: https://docs.python.org/3/library/sqlite3.html